



Assignment No. 01

Computer Organization & Assembly Language (EE-2003)

ID: 23I-0617

NAME: M. Abdullah Siddiqui

SECTION: F

Read the Instructions Carefully

- ❖ Assignments are designed for your learning copying will lead to plagiarism. Practicing the questions yourself will **help you in your exams**. Fill the **memory and register tables** in the word file first **before running the respective programs in visual studio**.
- ❖ The filled memory should represent the memory status of the **executed program**.
- ❖ Memory and register values should be in **hexadecimal format**.
- ❖ Each row in the memory table consists of **16 columns, each of 1 byte**. The row indicates the starting offset. Both the starting offset and the columns are represented with hexadecimal values.
- ❖ Your assignment marks will be calculated from your **submission (60%) + Quiz (30%) + Sessional-I (10%)**
- ❖ You are supposed to debug each question in Visual Studio after filling the tables.
- ❖ You are supposed to submit a **SNAPSHOT** of your visual studio debugging screen, such that it shows the status of registers and memory after debugging. Your submission must include snapshots of each question with a **visible time and date of your computer**. The snapshots for each question must be attached in the word file under respective questions. Remember each snapshot **has marks**.
- ❖ Submit a **PDF** File with the naming format **23i-1234_Section_A1**
- ❖ If you find any **ERROR** in the program highlight the error and write the justification for the error. You must understand ERROR carefully. After identifying the error, you will ignore it by commenting it out and running the program. Also, the register values corresponding to the **erroneous statements** must be left blank. For example:

```
; mov al, 256 ERROR: 256 is out of range for an 8-bit register like `AL`. It should be within 0-255.
```
- ❖ Data Declarations highlighted in yellow must be replaced as instructed below:

- ✓ 'F1' represents the first character of your first name, 'F2' represents the second character, 'F3' and 'F4' represent the third and the fourth characters of your first name, respectively. If your first name has less characters, then repeat the previous character(s) as needed.
- ✓ 'L1' represents the first character of your last name, 'L2' represents the second character, 'L3' and 'L4' represent the third and the fourth characters of your last name, respectively. If your last name has less characters, then repeat the previous character(s) as needed.
- ✓ 'R' represents the numeric part of your roll number. For example, if your roll number is 23i-1234, then 'R' is 1234. The base for 'R' is represented as (base), where the base can be 'h' for hexadecimal, 'd' for decimal, 'b' for binary, and 'o' for octal. You must first treat 'R' as a decimal number, then convert it into the specified base before substituting the value.

When R is 1234, then $R(d) = 1234d$

$R(h) = 04D2h$

$R(o) = 2322o$

$R(b) = 010011010010b$

- ✓ 'R1' and 'R2' represent the first and the second part of 'R', respectively. For example, if 'R' is 1234, then 'R1' is 12 and 'R2' is 34. The base for 'R1' and 'R2' are represented as (base), where the base can be 'h' for hexadecimal, 'd' for decimal, 'b' for binary, and 'o' for octal. You must first treat 'R1' and 'R2' as decimal numbers, then convert them into the specified bases before substituting the values.
- ✓ 'RR' represents the concatenated roll number. For example, if your roll number is 23i-1234 then 'RR' is 12341234. The "base" rules defined above also apply here.

- Consider the following Data declaration. Fill in the given Memory for following declaration.
Look for errors if any.

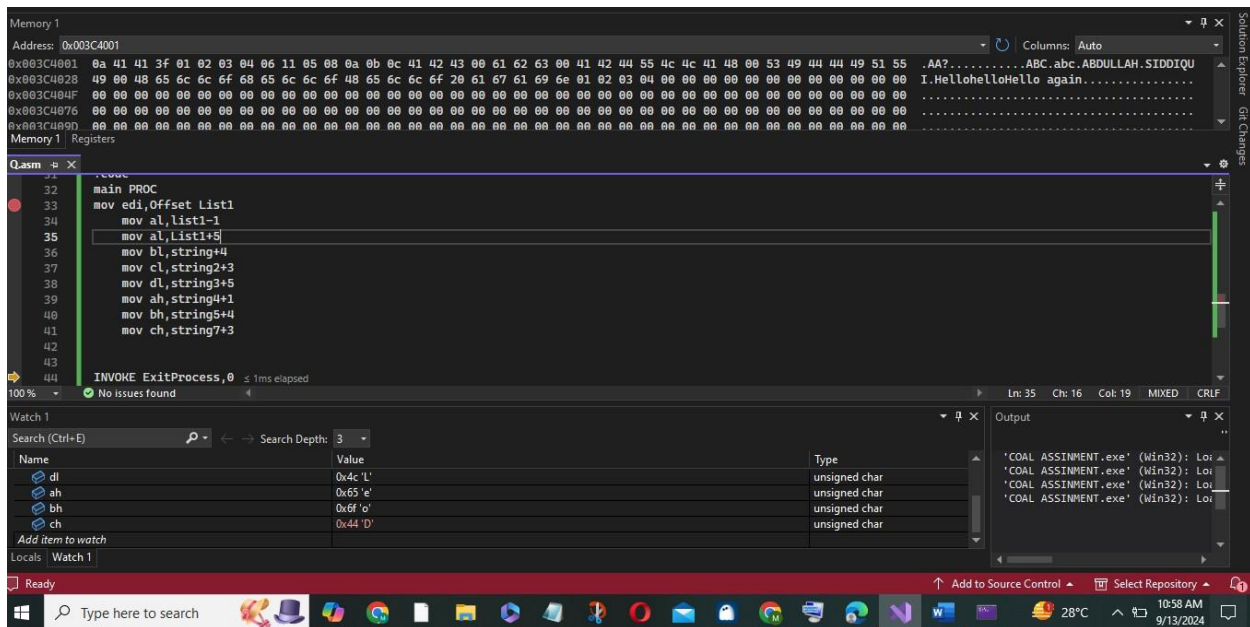
ASM Program																
<pre> .data valb_1 BYTE 'A' valb_2 BYTE 1111d ;Error Incompatible valb_3 DB +127 valb_3a DB 256 ;Error Incompatible valb_4 BYTE -128 valb_4a BYTE -129 valb_4b SBYTE -128 valb_5 BYTE 255 valb1_6 SBYTE -129 valb1_7 SBYTE +127 valb_8 SBYTE 'B' valb1_9 SBYTE -Ah .code main Proc mov edi,offset valb_1 mov al,valb_1 main ENDP END main </pre>																
MEMORY																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00400000	41	7F	80	7F	80	FF	7F	7F	42	F6						

The screenshot shows the Visual Studio Code interface with the assembly program loaded. The memory dump window displays the memory addresses and their corresponding values, which match the values in the table above. The Watch window shows the variable 'al' with the value 200 (hex 00C8). The Output window shows the execution output.

2. Consider the following Data declaration. Fill in the given Memory and register values. **Look for errors if any.**

DATA																																
<pre>.data DB -5 ;No Identifier Error List1 DB 10, 041h , 'A', 00111111b List2 BYTE 1,2,3,4 BYTE 0110b,10001b,5 BYTE 8,10,11,12 string BYTE 'ABC',0 string2 BYTE "abc",0 string3 BYTE 'ABDULLAH',0 string7 db "SIDDIQUI",0 string4 DB \ ;\ line continuation character "Hello" string5 b \ yte "hello" string5 \ DB "Hello again" list3 db 1,2, \ 3,4</pre>																																
CODE											REGISTER																					
<pre>.code main Proc mov edi,Offset List1 mov al,list1-1 mov al,List1+5 mov bl,string+4 mov cl,string2+3 mov dl,string3+5 mov ah,string4+1 mov bh,string5+4 mov ch,string7+3 main ENDP END main</pre>											<table><tr><td>ED</td><td>Add of</td></tr><tr><td>I</td><td>List1</td></tr><tr><td>AL</td><td>FB</td></tr><tr><td>AL</td><td>02</td></tr><tr><td>BL</td><td>61</td></tr><tr><td>CL</td><td>00</td></tr><tr><td>DL</td><td>4C</td></tr><tr><td>AH</td><td>6F</td></tr><tr><td>BH</td><td>48</td></tr><tr><td>CH</td><td>44</td></tr></table>		ED	Add of	I	List1	AL	FB	AL	02	BL	61	CL	00	DL	4C	AH	6F	BH	48	CH	44
ED	Add of																															
I	List1																															
AL	FB																															
AL	02																															
BL	61																															
CL	00																															
DL	4C																															
AH	6F																															
BH	48																															
CH	44																															
MEMORY																																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F																

00404000	FB	0A	41	41	3F	01	02	03	04	06	11	05	08	0A	0B	0C
00404010	41	42	43	00	61	62	63	00	41	42	44	55	4C	4C	41	48
00404020	00	53	49	44	44	49	51	55	49	00	48	65	6C	6C	6F	68
00404030	65	6C	6C	6F	48	65	6C	6C	6F	20	61	67	61	69	6E	01
00404040	02	03	04													
00404050																
00404060																
00404070																



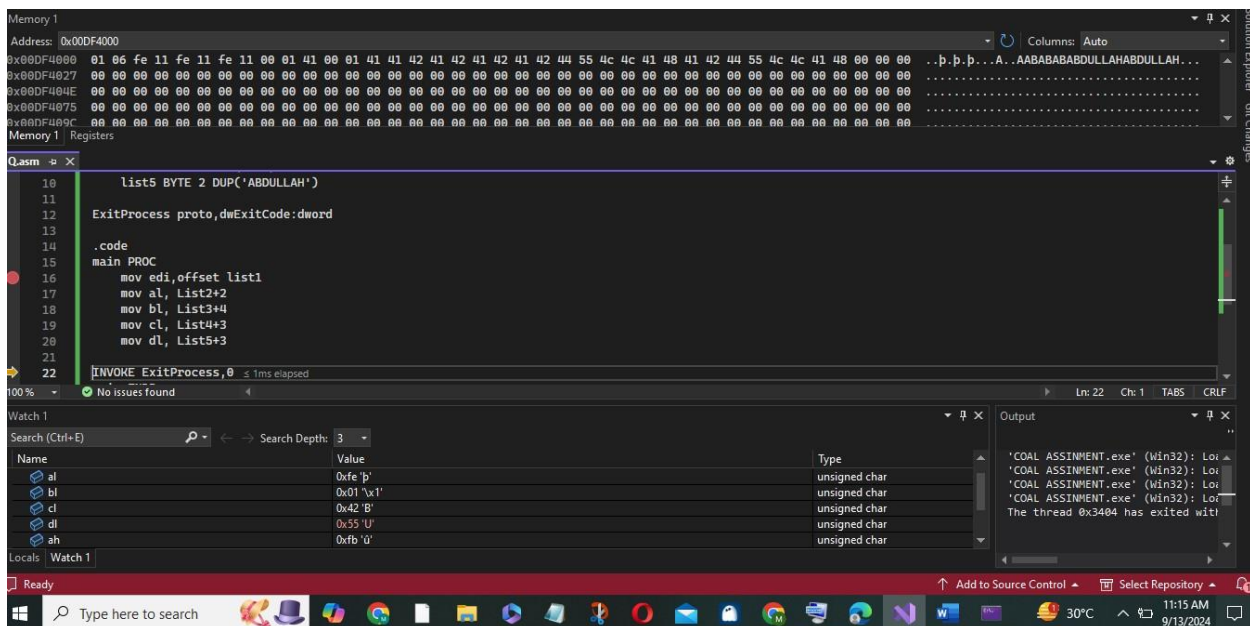
3. Consider the following Data declaration. Fill in the given Memory and register values. **Look for errors if any.**

DATA	
<pre> .data list1 BYTE 1,0110b list2 BYTE 3 DUP(-2,10001b) list3 DB 2 DUP(?,1, 'A') list4 BYTE 3 DUP('AB') list5 BYTE 2 DUP('ABDULLAH') </pre>	
CODE	REGISTER
<pre> .code main Proc </pre>	

<pre> mov edi,offset list1 mov al, List2+2 mov bl, List3+4 mov cl, List4+3 mov dl, List5+3 main ENDP END main </pre>	EDI	Add of list1
	AL	FE
	BL	01
	CL	42
	DL	55

MEMORY

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00404000	L1 01	06	L2 FE	11	FE	11	FE	11	L3 00	01	41	00	01	41	L4 41	42
00404010	41	42	41	42	L5 41	42	44	55	4C	4C	41	48	41	42	44	55
00404020	4C	4C	41	48												



4. Consider the following Data declaration. Fill in the given Memory and register values. **Look for errors if any.**

DATA
<pre> .data word1 DW 1 word2 DW -1 word3 SWORD -1 word4 SWORD +1 word5 DW 'AB' word6 DW 'ABC' </pre>

list1 DW 2 DUP(0269h) list2 DW 11510,2 list3 WORD 3 DUP(0) list4 WORD 2 DUP(0AB12H) List5 WORD 01F2BH,01101011001011b, 617 list6 DW 3 DUP('AB')																
CODE											REGISTER					
<pre>.code main Proc mov edi,offset list1 mov al, word1 mov bx, List1+4 mov cx, List2+1 mov dx, List4+2 mov ax, List6+1 mov cx, word5+2 mov cx, word5+4 main ENDP END main</pre>											ED I	Add of list1				
												Error Incompa tible				
											AL	0269				
											BX	0202				
											CX	AB12				
											DX	4241				
											AX	0269				
											CX	0269				
											MEMORY					
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00404000	W1 01	00	W2 FF	FF	W3 FF	FF	W4 01	00	W5 42	41	L1 69	02	69	02	L2 69	02
00404010	02	00	L3 00	00	00	00	00	00	L4 12	AB	12	AB	L5 2B	1F	CB	1A
00404020	69	02	L6 42	41	42	41	42	41								
00404030																

Memory 1

Address: 0x00AD4000

0x00AD4000

01 00 ff ff ff ff 01 00 42 41 69 02 69 02 69 02 02 00 00 00 00 00 00 00 12 ab 12 ab 2b 1f cb 1a 69 02 42 41 42 41 42

..yyy..BAi.i.i.....«.«.È.i.BABAB

0x00AD4027

41 00

A.....

0x00AD404E

00 00

.....

0x00AD4075

00 00

.....

0x00AD409C

00 00

.....

Memory 1 Registers

Qasm

24

;mov al, word1

25

mov bx, List1+4

26

mov cx, List2+1

27

mov dx, List4+2

28

mov ax, List6+1

29

mov cx, word5+2

30

mov cx, word5+4

31

32

33

INVOKE ExitProcess,0 < 1ms elapsed

34

main ENDP

35

END main

100% No issues found

Watch 1

Search (Ctrl+E)

Name

Value

Type

ax

0x4241

unsigned short

bx

0x0269

unsigned short

cx

0x0269

unsigned short

dx

0xab12

unsigned short

Add item to watch

Locals Watch 1

Output

'COAL ASSIGNMENT.exe' (Win32): Loi

'COAL ASSIGNMENT.exe' (Win32): Loi

'COAL ASSIGNMENT.exe' (Win32): Loi

'COAL ASSIGNMENT.exe' (Win32): Loi

Ready

Add to Source Control

Select Repository

Type here to search

30°C

11:38 AM 9/13/2024

- [illegible]

Memory 1
Address: 0x00994000
01 00 00 00 ff ff ff ff ff ff ff ff 00 00 01 00 59 58 00 00 48 47 46 45 78 56 34 12 cb 3a 0d 00 41 00 00 56 34 12
ab 56 34 12 ab 00
00
00
00 00

```

16
17 .code
18 main PROC
19     mov edi, Offset dword1
20     mov ebx, dword2
21     mov ecx, dword5
22     mov edx, dword6
23     mov eax, List1+4
24     mov ebx, List1+2
25     mov ecx, List1+8
26     mov ecx, dword5+4
27
28     INVOKE ExitProcess, 0

```

Name	Value	Type
edi	0x00994000	unsigned int
al	0xcb 'E'	unsigned char
bl	0x12 'x12'	unsigned char
cl	0x48 'H'	unsigned char
dl	0x48 'H'	unsigned char

Output
"COAL ASSIGNMENT.exe" (Win32): Lo...
"COAL ASSIGNMENT.exe" (Win32): Lo...
"COAL ASSIGNMENT.exe" (Win32): Lo...
"COAL ASSIGNMENT.exe" (Win32): Lo...
The thread 0x5f04 has exited with...

Registers
EAX = 00003ACB EBX = 3456AB12 ECX = 45464748 EDX = 45464748 ESI = 00991805 EDI = 00994000 EIP = 0099183E ESP = 0079F94C EBP = 0079F958 EFL = 00000246

```

16
17 .code
18 main PROC
19     mov edi, Offset dword1
20     mov ebx, dword2
21     mov ecx, dword5
22     mov edx, dword6
23     mov eax, List1+4
24     mov ebx, List1+2
25     mov ecx, List1+8
26     mov ecx, dword5+4
27
28     INVOKE ExitProcess, 0

```

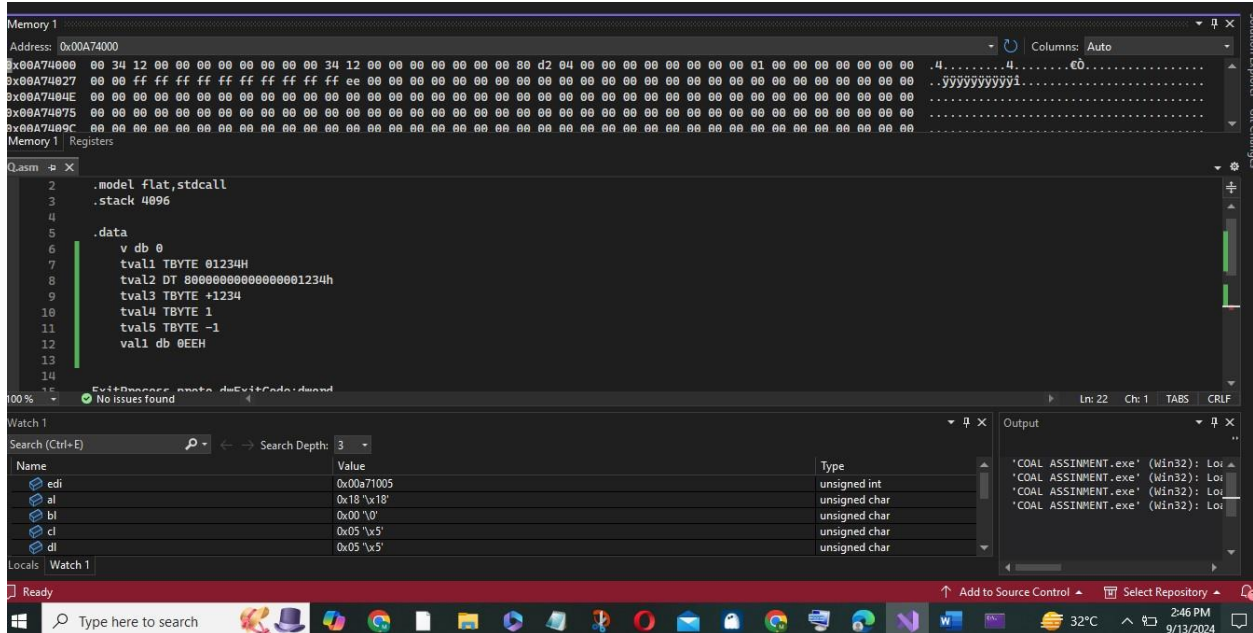
Name	Value	Type
edi	0x00994000	unsigned int
al	0xcb 'E'	unsigned char
bl	0x12 'x12'	unsigned char
cl	0x48 'H'	unsigned char
dl	0x48 'H'	unsigned char

Output
"COAL ASSIGNMENT.exe" (Win32): Lo...
"COAL ASSIGNMENT.exe" (Win32): Lo...
"COAL ASSIGNMENT.exe" (Win32): Lo...
"COAL ASSIGNMENT.exe" (Win32): Lo...
The thread 0x5f04 has exited with...

6. Consider the following Data declaration. Fill in the given Memory for following declaration. **Look for errors if any.**

DATA
<pre> .data v db 0 tval1 TBYTE 01234H tval2 DT 800000000000000001234h tval3 TBYTE +1234 tval4 TBYTE 1 tval5 TBYTE -1 val1 db 0EEH </pre>
MEMORY

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00404000	V 00	V1 34	12	00	00	00	00	00	00	00	00	V2 34	12	00	00	00
00404010	00	00	00	00	80	V3 D2	04	00	00	00	00	00	00	00	00	V4 01
00404020	00	00	00	00	00	00	00	00	00	V5 FF	FF	FF	FF	FF	FF	FF
00404030	FF	FF	FF	TV EE												

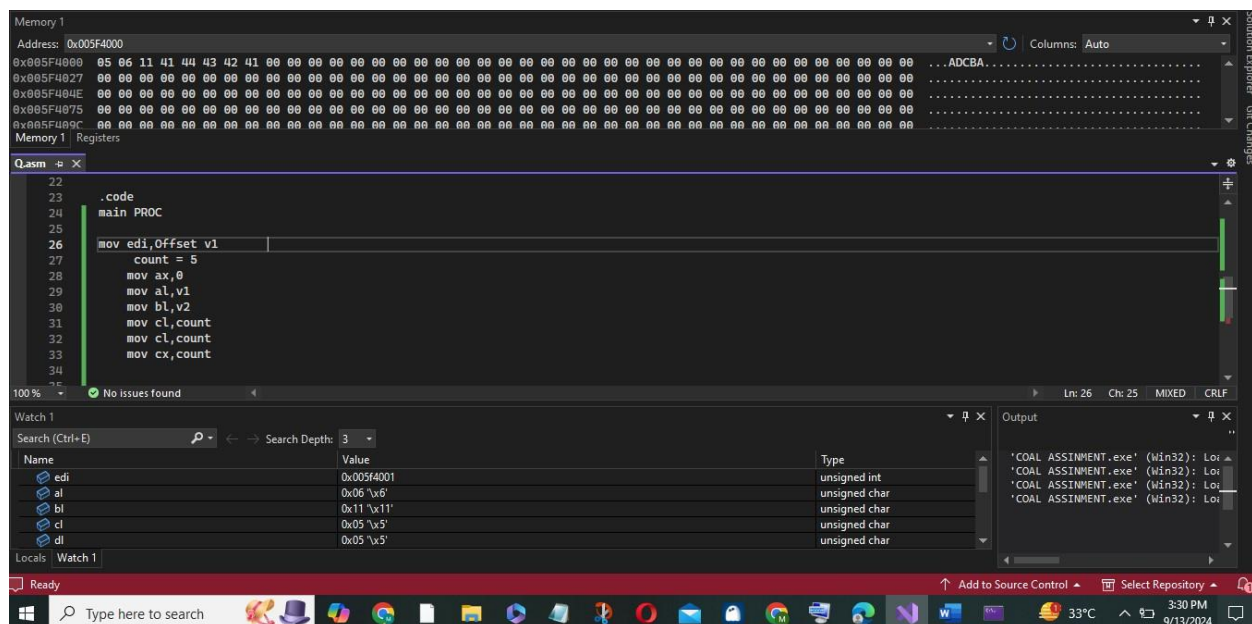


7. Consider the following Data declaration. Fill in the given Memory and register values. **Look for errors if any.**

DATA	
.data	
v3 db count	
count=2*2+4-2	
v1 db count	
count= 10001b	
v2 db count	
count2= 'A'	
v3 db count2	;ERROR REDEFINITION OF V3
count2='ABCD'	
count3= 'ABDUS'	;ERROR MAX SIZE IS 4 BYTES
v4 dd count2	
fcoun=2.5	;CANNOT BE A FRACTIONAL NUMBER
CountMax=4294967296	;ERROR MAX IS 2^32
CountMax=4294967294	

CODE													REGISTER				
<pre>.code main Proc mov edi,Offset v1 count = 5 mov ax,0 mov al,v1 mov bl,v2 mov cl,count count=257 ;too large mov cl,count mov cx,count main ENDP END main</pre>														ADD			
													EDI	OF V1			
													AX	0000			
													AL	06			
													BL	11			
													CL	05			
													CL				
													CX				

MEMORY																	
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
00404000	V3 05	V1 06	V2 11	V9 41	V4 44	43	42	41									
00404010																	



8. Consider the following Data declaration. Fill in the given Memory and register values. Look for errors if any.

DATA	
<pre> .data firstoffset dd \$ </pre>	<pre> ;Allocates 4 bytes to store address ;of firstoffset itself </pre>

```
;data is stored consecutively
```

10001b

CODE

```
main ENDP
END main
```

ESI

MEMORY

00404
020

Memory 1

Address: 0x00CE4004

Columns: Auto

Memory 1 Registers

Qasm

```

15  .code
16  main PROC
17  mov edi,Offset firstoffset
18  mov ah,list1+3
19  mov bh,list1size
20  mov cx,list2
21  mov dh,list2size
22  mov dl,list3size
23  mov esi,$
24
25
26
27  INVOKE ExitProcess,0

```

100% No issues found

Output

Show output from: Debug

COAL ASSIGNMENT.exe (Win32): Loaded 'D:\desktop\COAL ASSIGNMENT\Debug\COAL ASSIGNMENT.exe'. Sym

COAL ASSIGNMENT.exe (Win32): Loaded 'C:\Windows\System32\ntdll.dll'.

COAL ASSIGNMENT.exe (Win32): Loaded 'C:\Windows\System32\kernel32.dll'.

COAL ASSIGNMENT.exe (Win32): Loaded 'C:\Windows\System32\user32.dll'.

Watch 1

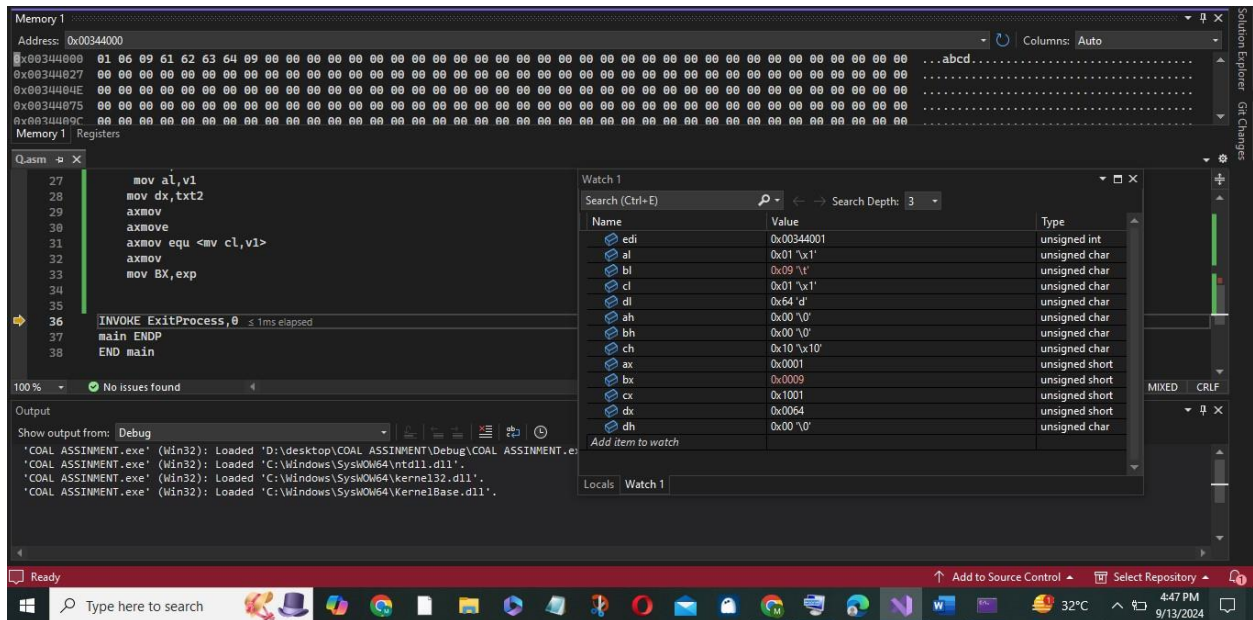
Search (Ctrl+E) Search Depth: 3

Name	Value	Type
edi	0x00CE4000	unsigned int
al	0x08 'b'	unsigned char
bl	0x00 '0'	unsigned char
cl	0x01 'x1'	unsigned char
dl	0x03 'x3'	unsigned char
ah	0x41 'A'	unsigned char
bh	0x04 'x4'	unsigned char
ch	0x00 '0'	unsigned char
ax	0x4108	unsigned short
bx	0x0400	unsigned short
cx	0x0001	unsigned short
dx	0x0603	unsigned short
dh	0x06 'x6'	unsigned char

Add item to watch

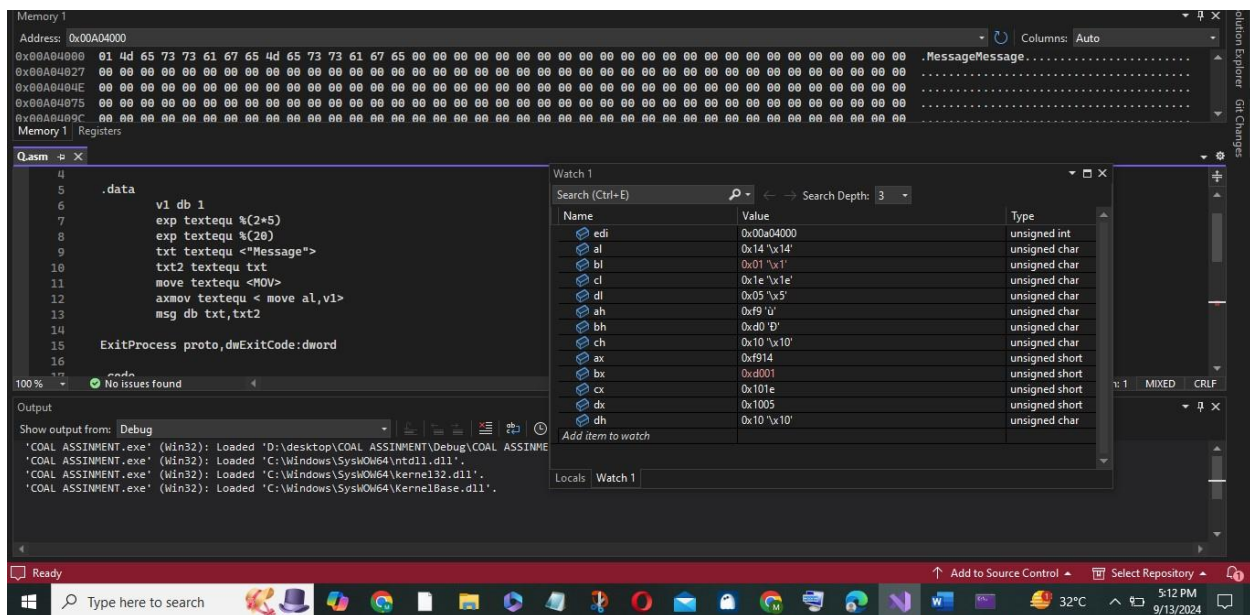
Locals: Watch 1

- [illegible]



10. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

DATA													
<pre> .data v1 db 1 exp textequ %(2*5) exp textequ %(20) txt textequ <"Message"> txt2 textequ txt move textequ <MOV> axmov textequ < move al,v1> msg db txt,txt2 </pre>													
CODE	REGISTER												
<pre> .code main Proc mov edi,Offset v1 mov al,v1 axmov mov al,exp exp textequ %(30) axmov textequ < move bl,v1> axmov mov Cl,exp main ENDP END main </pre>	<table border="1"> <tr> <td>EDI</td><td>Add of v1</td></tr> <tr> <td>AL</td><td>01</td></tr> <tr> <td>AL</td><td>01</td></tr> <tr> <td>AL</td><td>14</td></tr> <tr> <td>BL</td><td>01</td></tr> <tr> <td>CL</td><td>1E</td></tr> </table>	EDI	Add of v1	AL	01	AL	01	AL	14	BL	01	CL	1E
EDI	Add of v1												
AL	01												
AL	01												
AL	14												
BL	01												
CL	1E												

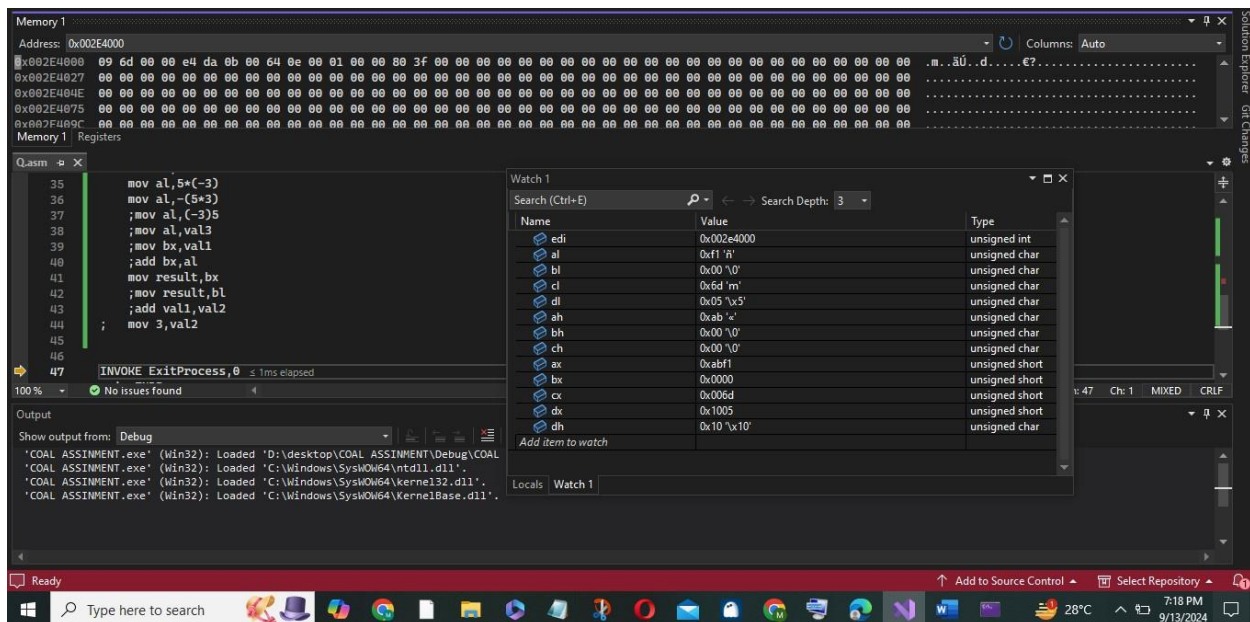


11. Consider the following Data declaration. Fill in the given Memory and register values. **Look for errors if any.**

DATA		
<pre> .data val1 db 1001b val2 db -10010011y val3 dw 1.0 val4 db -34o val5 dw 5732q val6 db 2.5 val7 db 1+'a'+4/2 val8 dw 5*3/4*2+8 val9 db -255 val4 dd 1.0 ;REDEFINITION?? wval WORD 0 result word ? </pre>		
CODE	REGISTER	
<pre> .code main Proc mov edi,Offset val1 mov ax,3.5 ;NO REAL NUMBER mov bl,val2*2 ;MUL NOT SUPPORT ON VARIABLES LIKE TIS mov al,2*(128+1) ;TOO BIG mov cl, val3-1 ;NOT COMPATIBLE mov cl,val1+val1a ;VAL1A NOT DEFINED mov al,-45o mov ax,'a' mov ax,'ab' mov ax,AB12h ;0 IN PREFIX IS IMP </pre>	EDI	Add of val1
	AX	
	BL	
	AL	
	CL	
	CL	
	AL	DB
	AX	0061
	AX	6162

<pre>mov al,5*(-3) mov al,-(5*3) mov al,(-3)5 ;NO OPERATOR mov al,val3 ;NOT COMPATIBLE mov bx,val1 ;NOT COMPATIBLE add bx,al ;NOT COMPATIBLE mov result,bx mov result,bl ;NOT COMPATIBLE add val1,val2 ;MEM TO MEM NOT ALLOW mov 3,val2 ;WRONG SYNTAX main ENDP END main</pre>													AX	AB12
													AL	F1
													AL	F1
													AL	
													AL	
													BX	
													BX	

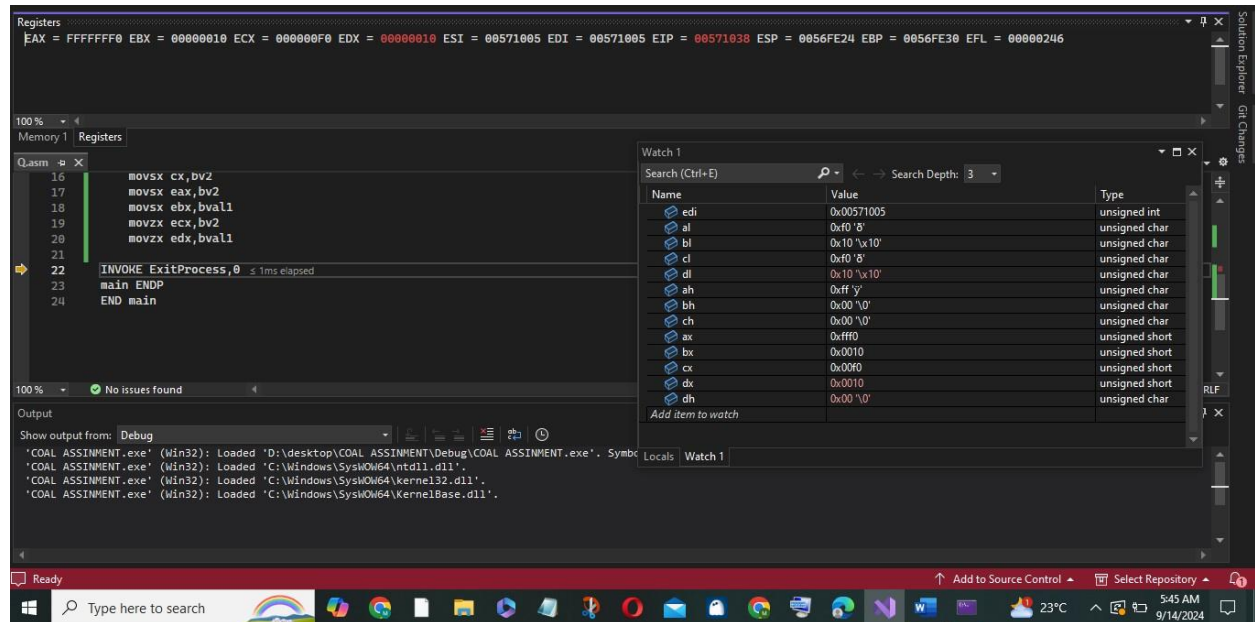
MEMORY																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00404000	V1 09	V2 6D	V3 00	00	V4 E4	V5 DA	0B	V6 00	V7 64	V8 0E	00	V9 01	VX 00	00	3F	80
00404010	Wv 00	00	RE 00	00												



12. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

DATA	
<pre> .data bval1 BYTE 16 bv2 BYTE -16 wval WORD 0 </pre>	
CODE	REGISTER

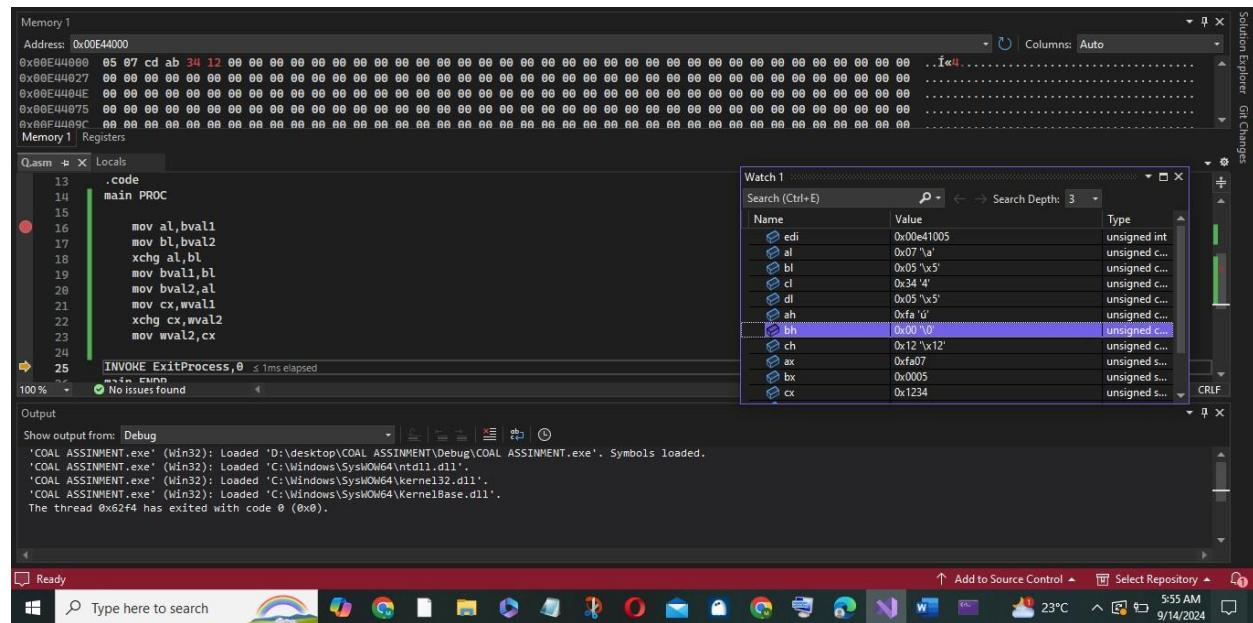
<pre> .code main Proc movsx bx,al movsx cx,bv2 movsx eax,bv2 movsx ebx,bval1 movzx ecx,bv2 movzx edx,bval1 main ENDP END main </pre>	BX	0000
	CX	FFF0
		FFFFFF
	EAX	F0
		0000001
	EBX	0
		000000
	ECX	F0
		0000001
	EDX	0



13. Consider the following Data declaration. Fill in the given Memory and register values. **Look for errors if any.**

DATA												
<pre>.data bval1 BYTE 5 bval2 BYTE 7 wval1 WORD 0ABCDH wval2 word 01234h</pre>												
CODE		REGISTER										
<pre>.code main Proc mov al,bval1 mov bl,bval2 xchg al,bl mov bval1,bl mov bval2,al mov cx,wval1 xchg cx,wval2</pre>		<table><tr><td>AL</td><td>05</td></tr><tr><td>BL</td><td>07</td></tr><tr><td>AL</td><td>07</td></tr><tr><td>CX</td><td>ABCD</td></tr><tr><td>CX</td><td>1234</td></tr></table>	AL	05	BL	07	AL	07	CX	ABCD	CX	1234
AL	05											
BL	07											
AL	07											
CX	ABCD											
CX	1234											

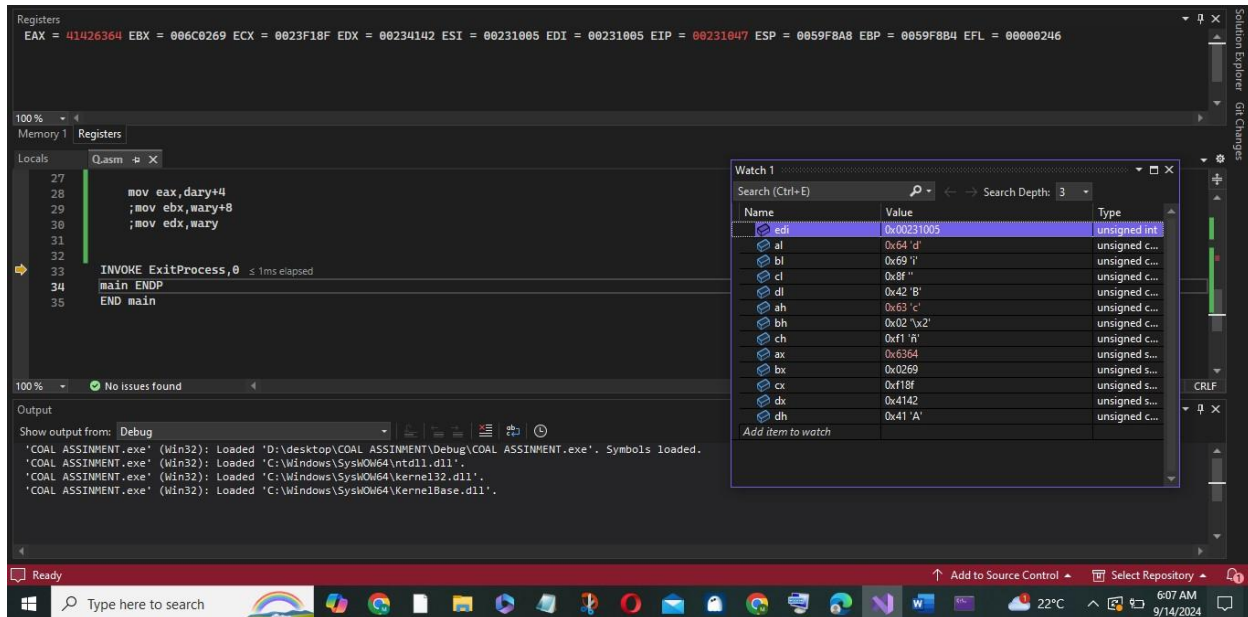
mov wval2,cx																
main ENDP END main																
MEMORY																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00404000	V1 05	V2 07	Wv CD	AB	Wv 34	12										



14. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

DATA	
<pre> .data bary BYTE 0ABh,0450,11h,25T,50q wary WORD 34560,0269h,'cd' Dary dd 061706170,'ABcd','SIDD',0AB12CD34h Qary dq 12345678abcdefh,'AbCd' </pre>	
CODE	REGISTER
<pre> .code main Proc mov al,bary mov bl,bary+1 mov cl,bary+2 mov dl,bary+3 mov ax,wary mov bx,wary+2 </pre>	AL
	AB
	BL
	25
	CL
	11
	DL
	19
	AX
	072E
	BX
	0269
	CX
	F18F

mov cx,wary+6		DX	4142
mov dx,wary+12			41426
		EAX	364
mov eax,dary+4		EBX	
mov ebx,wary+8	;Incompatible	EDX	
mov edx,wary	;Incompatible		
main ENDP			
END main			



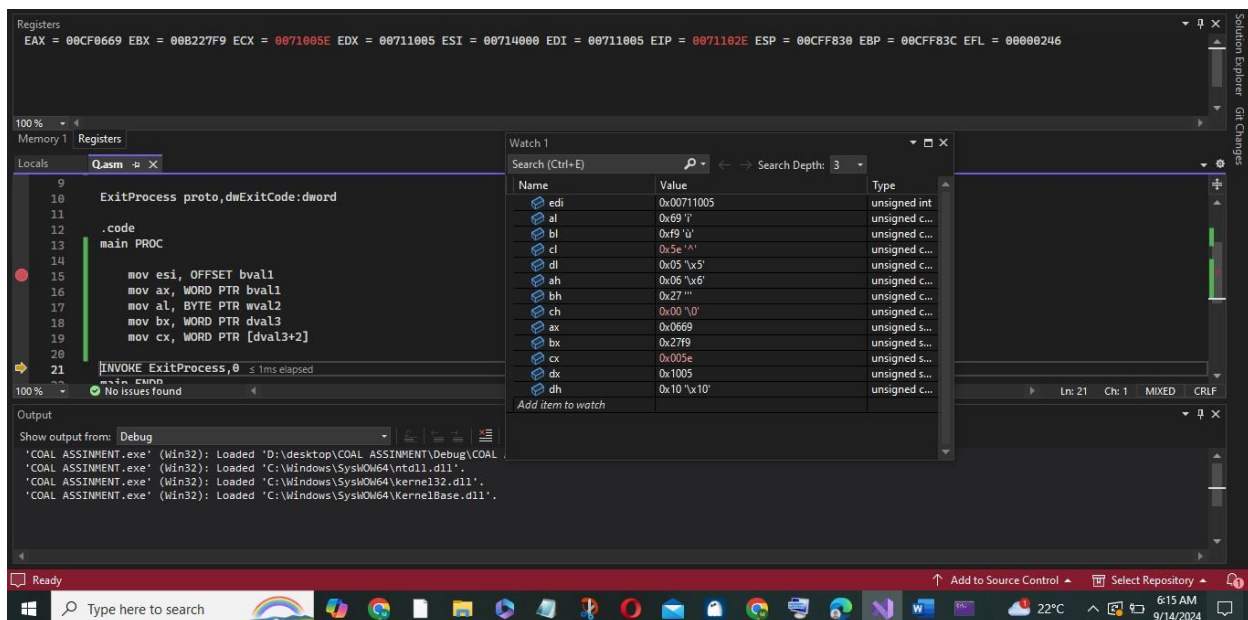
15. Consider the following Data declaration. Fill in the given Memory for following declaration. **Look for errors if any.**

DATA															
<pre> .data bval1 DB 1,2, 17d Align 2 wval2 DW 1001101001b bval2 db 6d Align dword dval3 DD 5E27F9h Align 4 val4 DB 'A' ALIGN WORD val5 DB 'S' pary DD bval1 </pre>															
MEMORY															
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	F

00404000	01	02	11	00	69	02	06				F9	27	5E	00		
00404010	41			S												
00404020																

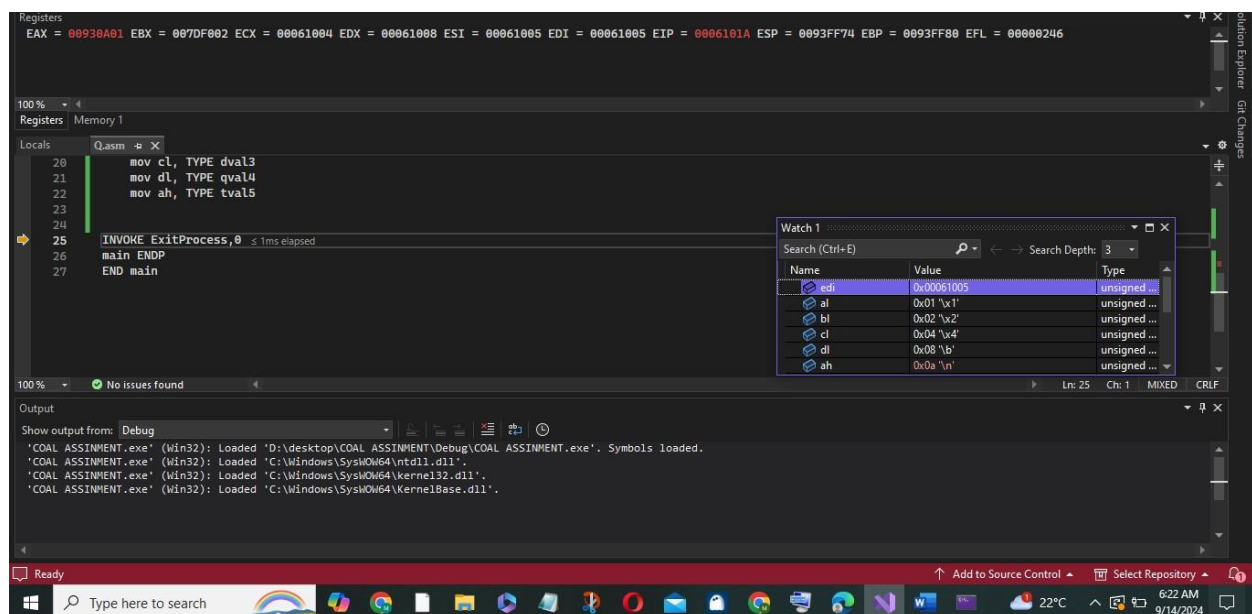
16. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

DATA											
<pre>.data bval1 DB 034h, 0110b wval2 DW 0269h dval3 DD 101111000100111111111001b</pre>											
CODE	REGISTER										
<pre>.code main Proc mov esi, OFFSET bval1 mov ax, WORD PTR bval1 mov al, BYTE PTR wval2 mov bx, WORD PTR dval3 mov cx, WORD PTR [dval3+2] main ENDP END main</pre>	<table> <tr> <td>ESI</td><td>Add of bval1</td></tr> <tr> <td>AX</td><td>0634</td></tr> <tr> <td>AL</td><td>69</td></tr> <tr> <td>BX</td><td>27F9</td></tr> <tr> <td>CX</td><td>005E</td></tr> </table>	ESI	Add of bval1	AX	0634	AL	69	BX	27F9	CX	005E
ESI	Add of bval1										
AX	0634										
AL	69										
BX	27F9										
CX	005E										



17. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

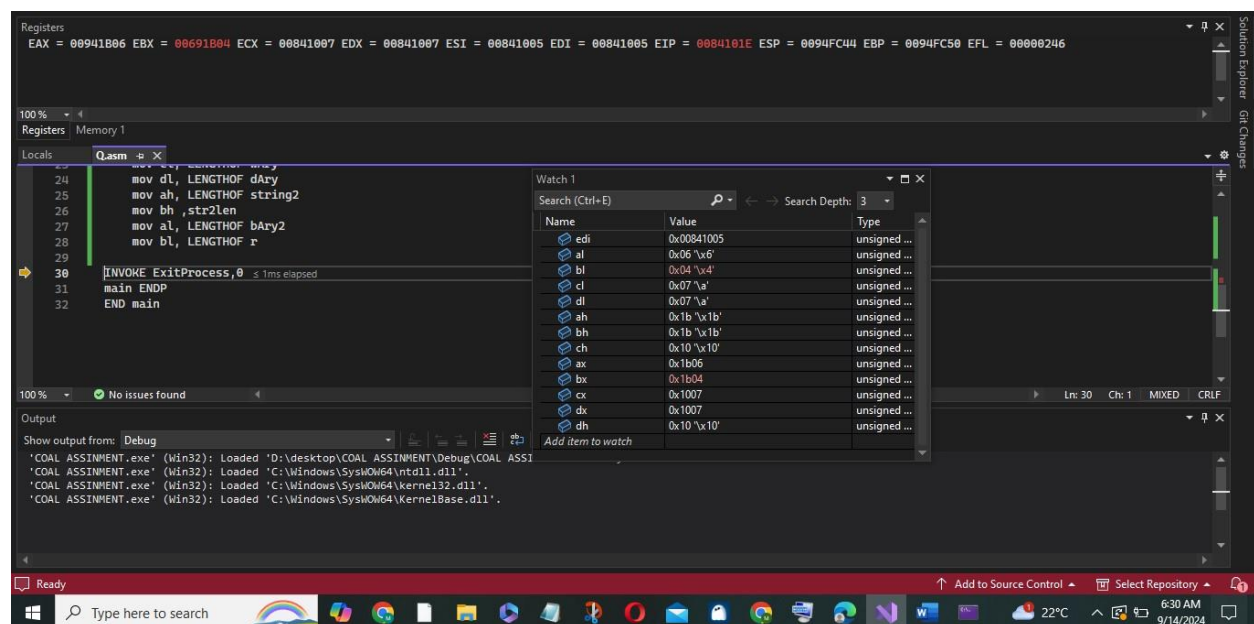
DATA											
<pre> .data bval1 DB 'A' wval2 DW 0ABCDH,'B' dval3 DD 5E27F9h qval4 QWORD 01243567812435678H tval5 tbyte -1 </pre>											
CODE	REGISTER										
<pre> .code main Proc mov al, TYPE bval1 mov bl, TYPE wval2 mov cl, TYPE dval3 mov dl, TYPE qval4 mov ah, TYPE tval5 main ENDP END main </pre>	<table border="1"> <tr><td>AL</td><td>01</td></tr> <tr><td>BL</td><td>02</td></tr> <tr><td>CL</td><td>04</td></tr> <tr><td>DL</td><td>08</td></tr> <tr><td>AH</td><td>0A</td></tr> </table>	AL	01	BL	02	CL	04	DL	08	AH	0A
AL	01										
BL	02										
CL	04										
DL	08										
AH	0A										



18. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

DATA
<pre> .data bAry BYTE 010H,020h,030H wAry WORD 5 DUP(?),0 ,0 string1 BYTE 0FFh,0FFh dAry DWORD 2 DUP(3 DUP(0)),0 string2 BYTE 'ABCDEFGHIJKLMNOPQRSTUVWXYZ',0 str2len= \$-string2 </pre>

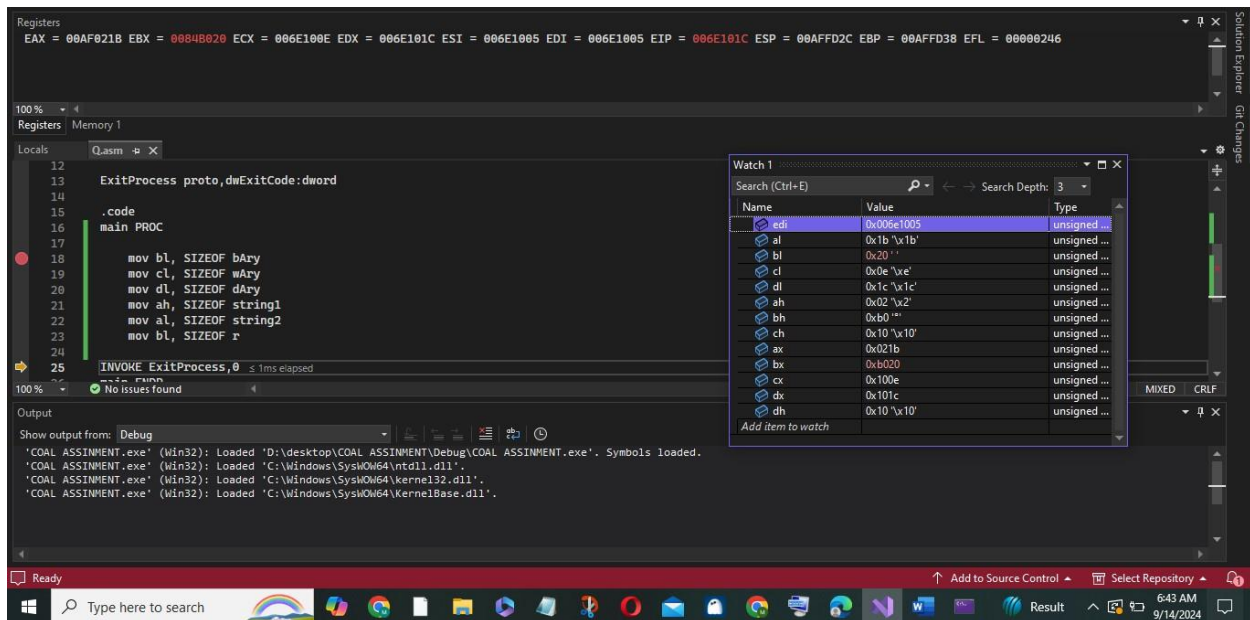
bAry2 BYTE 1,2,3, 4,5,6 BYte 4,5,6 r dq 2 dup(5),2 dup('abcd')															
CODE	REGISTER														
<pre> .code main Proc mov bl, LENGTHOF bAry mov cl, LENGTHOF wAry mov dl, LENGTHOF dAry mov ah, LENGTHOF string2 mov bh, str2len mov al, LENGTHOF bAry2 mov bl, LENGTHOF r main ENDP END main </pre>	<table border="1"> <tr><td>BL</td><td>03</td></tr> <tr><td>CL</td><td>07</td></tr> <tr><td>DL</td><td>07</td></tr> <tr><td>AH</td><td>1B</td></tr> <tr><td>BH</td><td>1B</td></tr> <tr><td>AL</td><td>06</td></tr> <tr><td>BL</td><td>04</td></tr> </table>	BL	03	CL	07	DL	07	AH	1B	BH	1B	AL	06	BL	04
BL	03														
CL	07														
DL	07														
AH	1B														
BH	1B														
AL	06														
BL	04														



19. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

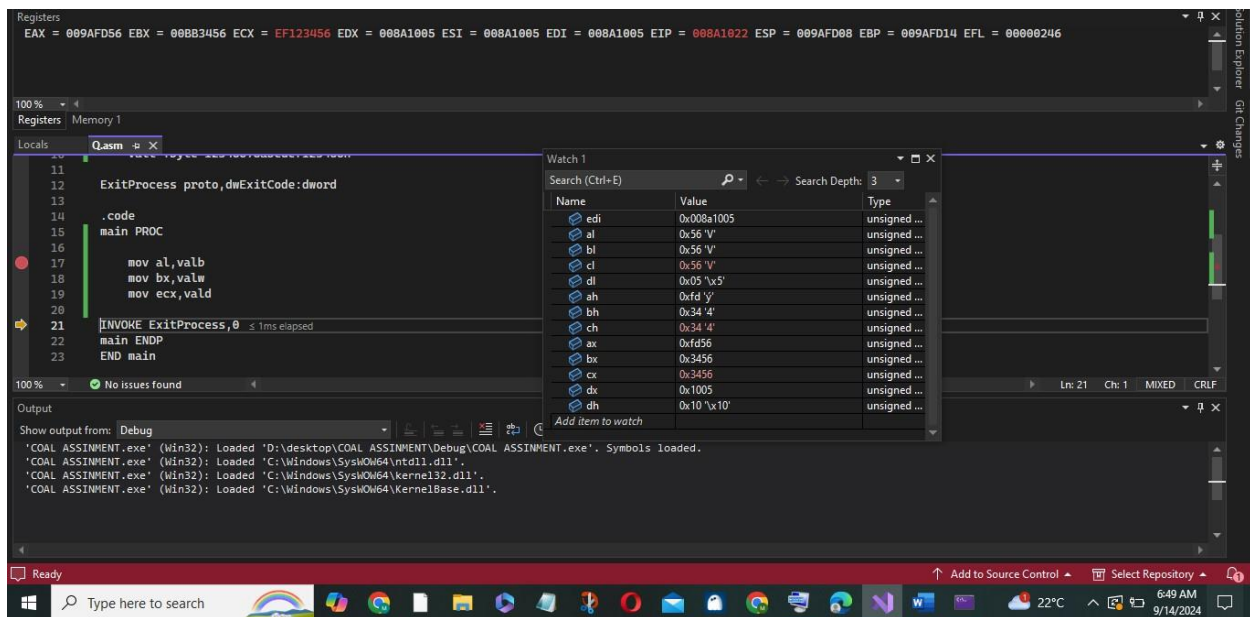
DATA			
<pre> .data bAry BYTE 010H,020h,030H wAry WORD 5 DUP(?),0 ,0 string1 BYTE 0FFh,0FFh dAry DWORD 2 DUP(3 DUP(0)),0 string2 BYTE 'ABCDEFGHIJKLMNOPQRSTUVWXYZ',0 r dq 2 dup(5),2 dup('abcd') </pre>			
CODE	REGISTER		
<pre> .code main Proc </pre>	<table border="1"> <tr><td>BL</td><td>03</td></tr> </table>	BL	03
BL	03		

mov bl, sizeof bAry	CL	0E
mov cl, sizeof wAry	DL	1C
mov dl, sizeof dAry	AH	02
mov ah, sizeof string1	AL	1B
mov al, sizeof string2	BL	20
mov bl, sizeof r		
main ENDP		
END main		



20. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

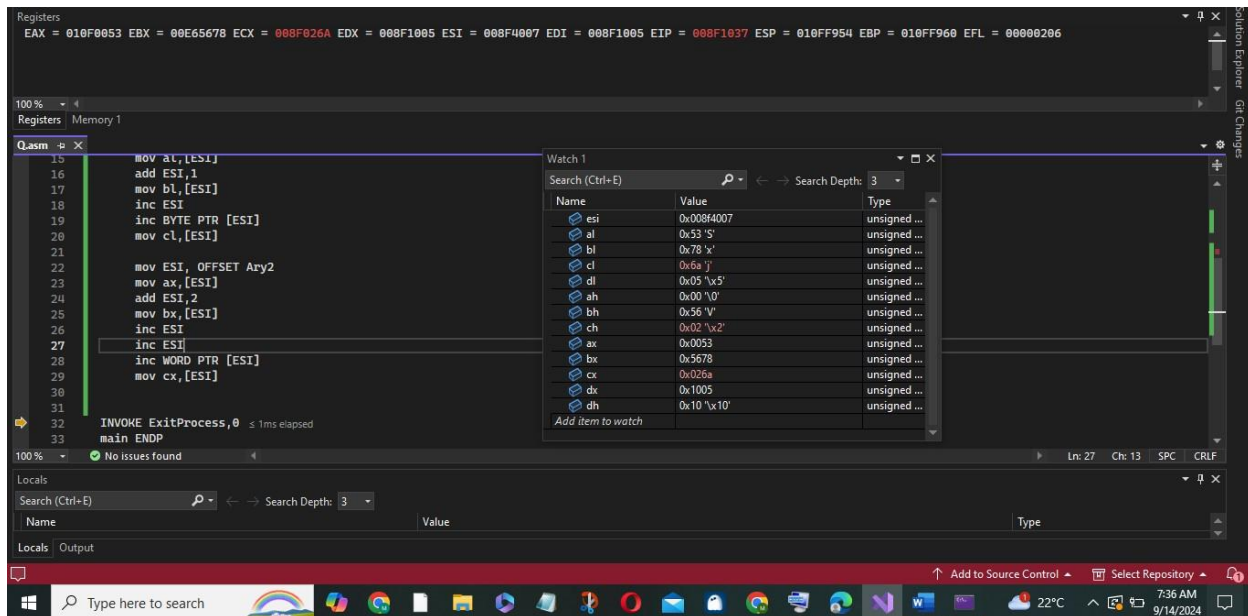
DATA							
<pre>.data valb LABEL Byte valw LABEL WORD vald LABEL DWORD valq LABEL dq valt Tbyte 12345678abcdef123456h</pre>							
CODE	REGISTER						
<pre>.code main Proc mov al, valb mov bx, valw mov ecx, vald main ENDP END main</pre>	<table> <tr> <td>AL</td><td>56</td></tr> <tr> <td>BX</td><td>3456</td></tr> <tr> <td>ECX</td><td>EF123456</td></tr> </table>	AL	56	BX	3456	ECX	EF123456
AL	56						
BX	3456						
ECX	EF123456						



21. Consider the following Data declaration. Fill in the given Memory and register values. **Look for errors if any.**

DATA																					
<pre> .data Ary1 BYTE 'A',29, 21o ary2 WORD 'S',05678h, 1001101001b </pre>																					
CODE	REGISTER																				
<pre> .code main Proc mov ESI, OFFSET Ary1 mov al,[ESI] add ESI,1 mov bl,[ESI] inc ESI inc BYTE PTR [ESI] mov cl,[ESI] mov ESI, OFFSET Ary2 mov ax,[ESI] add ESI,2 mov bx,[ESI] inc ESI inc ESI inc WORD PTR [ESI] mov cx,[ESI] </pre>	<table border="1"> <tr><td>ESI</td><td>00404000</td></tr> <tr><td>AL</td><td>41</td></tr> <tr><td>ESI</td><td>00404001</td></tr> <tr><td>BL</td><td>1D</td></tr> <tr><td>ESI</td><td>00404002</td></tr> <tr><td>CL</td><td>12</td></tr> <tr><td>ESI</td><td>00404003</td></tr> <tr><td>AX</td><td>0053</td></tr> <tr><td>ESI</td><td>00404005</td></tr> <tr><td>BX</td><td>5678</td></tr> </table>	ESI	00404000	AL	41	ESI	00404001	BL	1D	ESI	00404002	CL	12	ESI	00404003	AX	0053	ESI	00404005	BX	5678
ESI	00404000																				
AL	41																				
ESI	00404001																				
BL	1D																				
ESI	00404002																				
CL	12																				
ESI	00404003																				
AX	0053																				
ESI	00404005																				
BX	5678																				

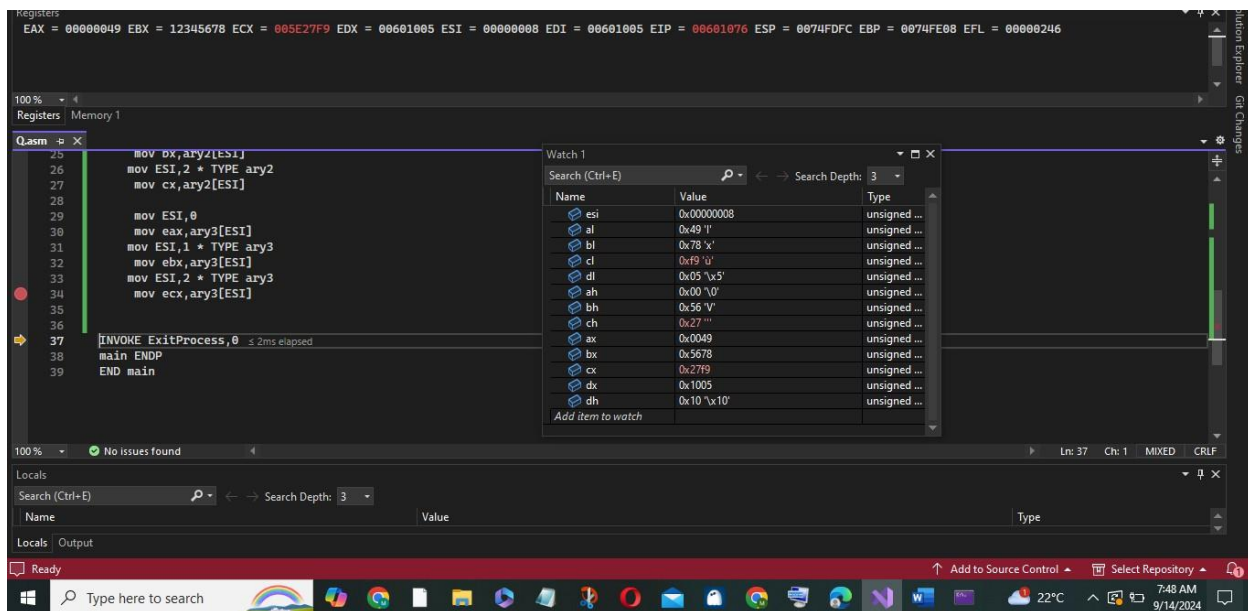
main ENDP END main												ESI 004040 007		CX 026A		
MEMORY																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00404000	A1 41	1D 11		A2 53	00 78		56	69	02							
00404010																
00404020																
00404030																



22. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

DATA					
<pre> .data Ary1 BYTE 'A',2, 11h,4 ary2 WORD 'S',05678h, 11510,5670 ary3 DWORD 'I',012345678h, 5E27F9h,5670 </pre>					
CODE	REGISTER				
<pre> .code main Proc mov esi,0 mov al,ary1[ESI] mov esi,1 * TYPE ary1 </pre>	<table border="1"> <tr> <td>ESI</td><td>00000</td></tr> <tr> <td>AL</td><td>41</td></tr> </table>	ESI	00000	AL	41
ESI	00000				
AL	41				

mov bl,ary1[ESI]	ESI	00001
mov esi,2 * TYPE ary1	BL	02
mov cl,ary1[ESI]	ESI	00002
	CL	11
mov esi,0	ESI	00000
mov ax,ary2[ESI]	AX	0053
mov esi,1 * TYPE ary2	ESI	00002
mov bx,ary2[ESI]	BX	5678
mov esi,2 * TYPE ary2	ESI	00004
mov cx,ary2[ESI]	CX	0269
	ESI	00000
mov esi,0		000000
mov eax,ary3[ESI]	EAX	49
mov esi,1 * TYPE ary3	ESI	00004
mov ebx,ary3[ESI]		123456
mov esi,2 * TYPE ary3	EBX	78
mov ecx,ary3[ESI]	ESI	00008
		005E27
main ENDP	ECX	F9
END main		



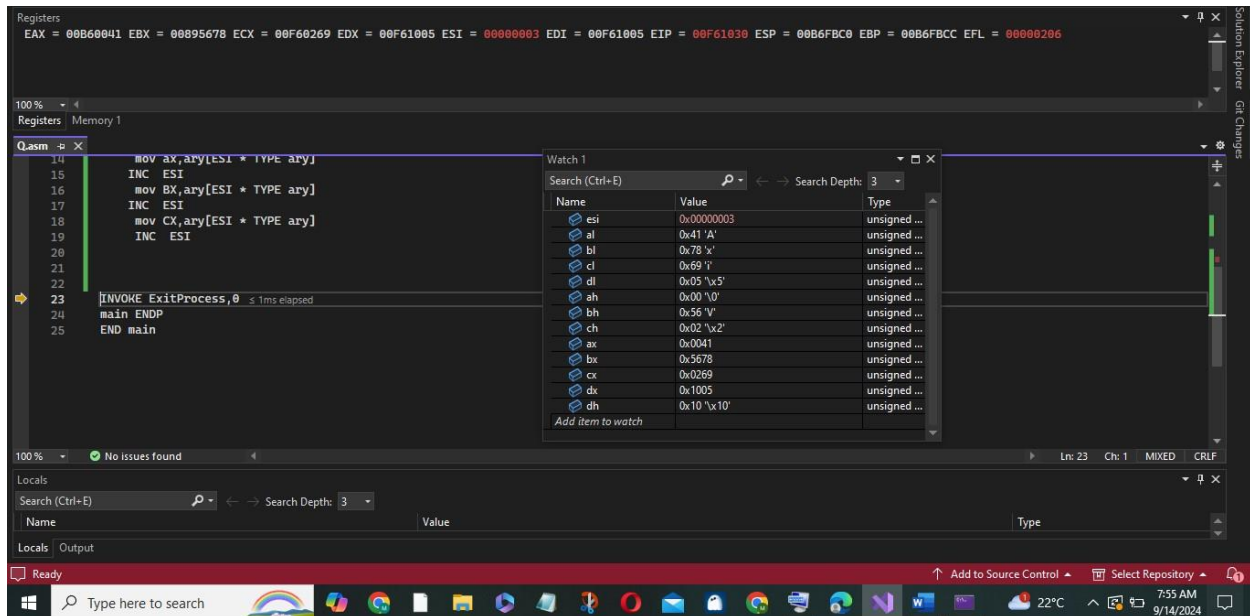
23. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

DATA	
<pre> .data ary WORD 'A',05678h, 1001101001b,5670 </pre>	
CODE	REGISTER

.code

```
main Proc
    mov ESI,0
    mov ax,ary[ESI * TYPE ary]
    INC ESI
    mov BX,ary[ESI * TYPE ary]
    INC ESI
    mov CX,ary[ESI * TYPE ary]
    INC ESI
main ENDP
END main
```

ESI	00000
AX	0041
ESI	00001
BX	5678
ESI	00002
CX	0269
ESI	00003



24. Write an assembly program that reverses the following string in the memory. Attach screenshot. Only 'jmp' and labels are allowed:

'!egaugnal ylbmessA eroda I'

.data

point label BYTE

arr BYTE '!egaugnal ylbmessA eroda I'

start DWORD 0

endd DWORD 0

.code

main PROC

mov eax, lengthof arr

mov endd, eax

SUB endd, 1

L1:

mov esi, start

mov edi, endd

mov al, [point + esi]

mov bl, [point + edi]

xchg [point + edi], al

xchg [point + esi], bl

ADD start, 1

SUB endd, 1

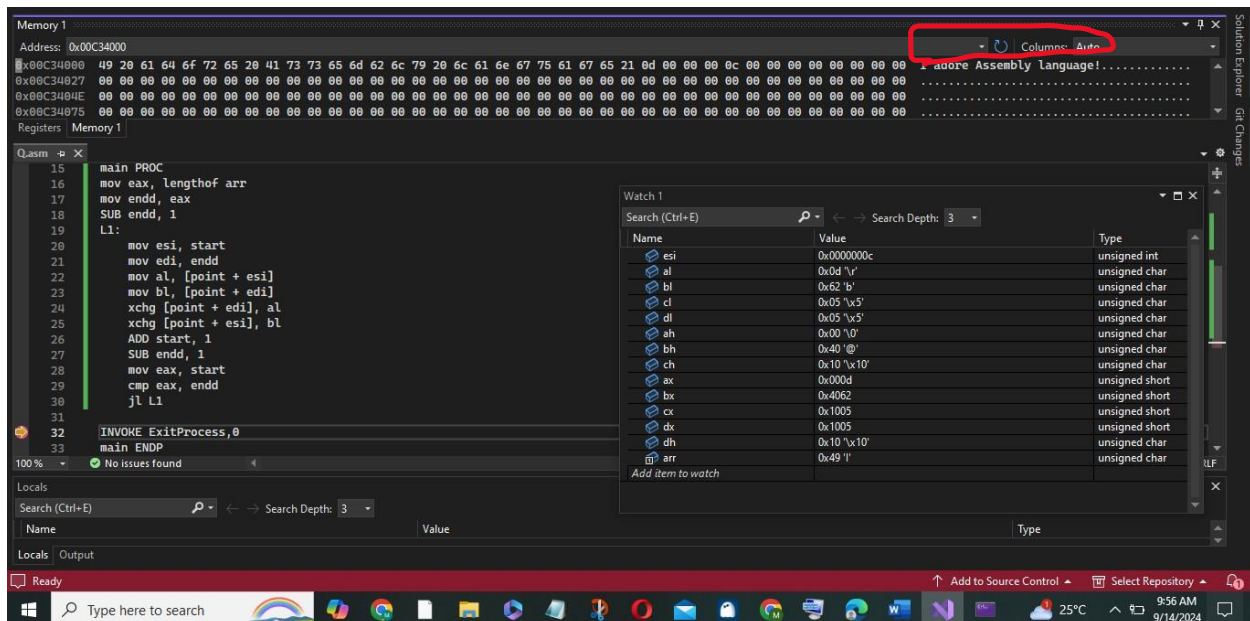
mov eax, start

cmp eax, endd

jl L1

main ENDP

END main



25. Implement following C code in assembly language. Attach screenshot. **You can use jmp/labels and loop:**

```

int si=0;
for(int a=0;a<=3;a++)
{
    for(int b=a;b<=8;b=b+2)
    {
        si++;
    }
}

```

.data

b DWORD 0

s BYTE 0

a DWORD 3

com DWORD 8

temp DWORD 0

.code

ADD a, 1

ADD com, 1

mov ecx, a

L1:

mov b, ecx

mov eax, a

mov temp, eax

SUB temp, ecx

mov eax, temp

mov b, eax

L2:

ADD s, 1

ADD b, 2

mov ebx, com

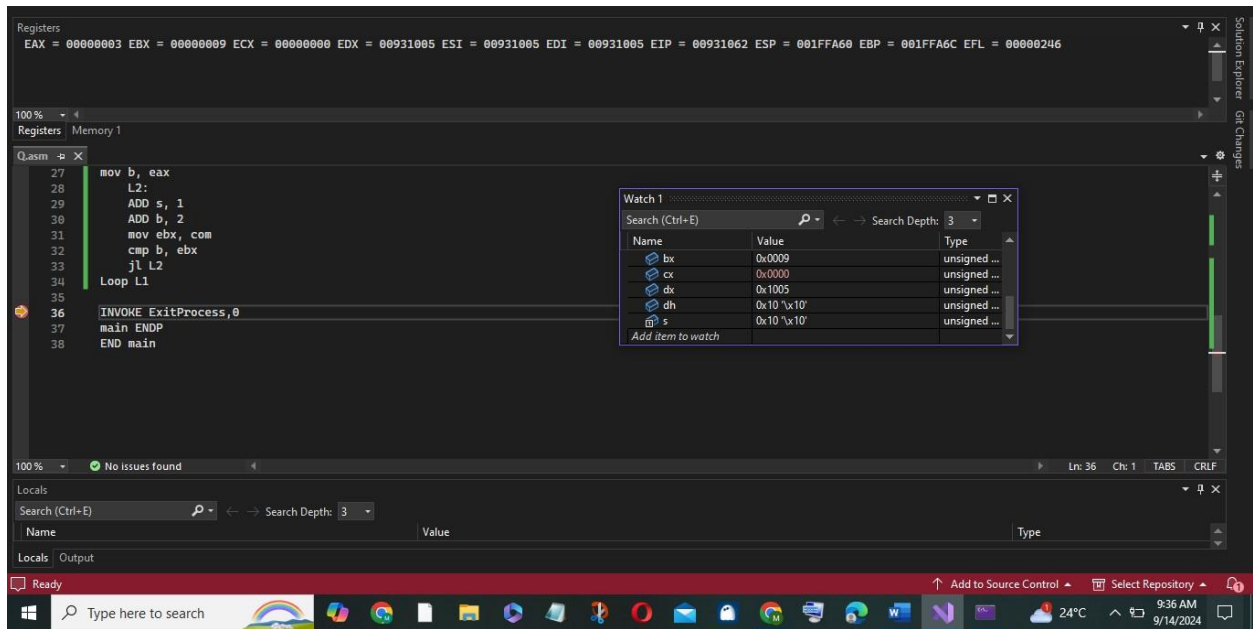
cmp b, ebx

j1 L2

Loop L1

main ENDP

END main



26. Write code that initializes the following array with the first 100 integers using INDIRECT ADDRESSING. You must upload a screenshot of memory. Only 'jmp' and labels are allowed:

```
ary dword 100 dup(0abcdef12h)
```

.data

```
ary DWORD 100 DUP(0abcdef12h)
```

```
cou DWORD 0
```

.code

```
main PROC
```

```
mov esi, OFFSET ary
```

```
L1:
```



```

mov eax, cou

mov [esi], eax

inc esi

inc esi

inc esi

inc esi

ADD cou, 1

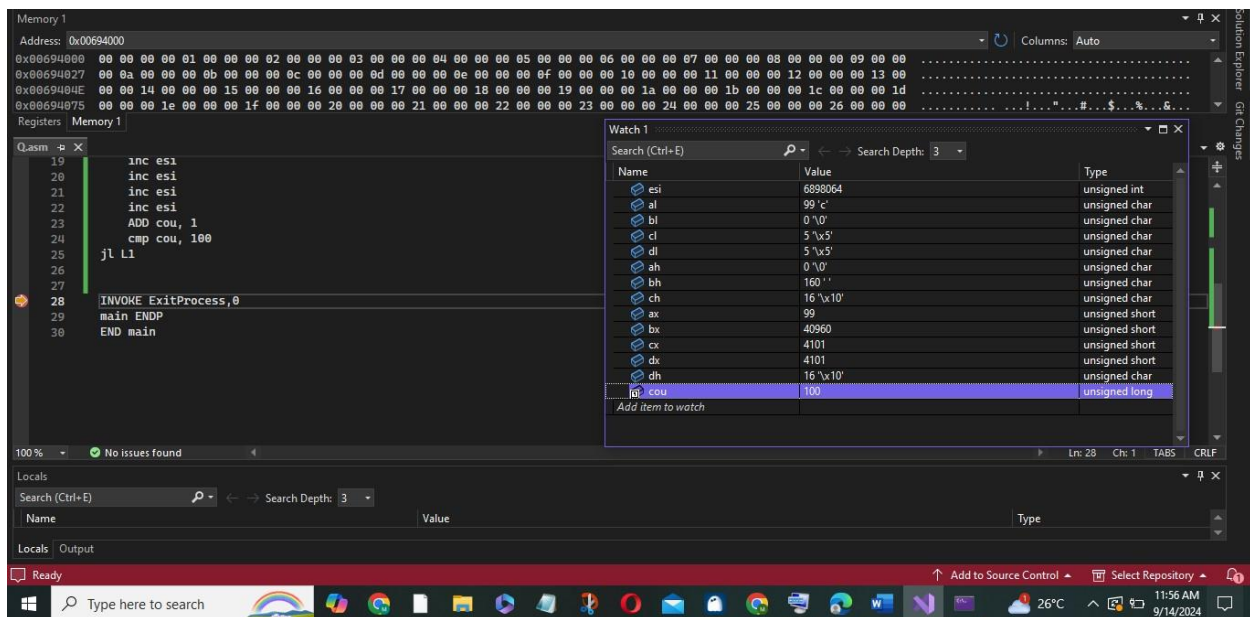
cmp cou, 100

jl L1

main ENDP

END main

```



27. Write an assembly program that converts your name from lowercase to uppercase. You must update the name in the memory and attach a screenshot of it. **You can use jmp/labels or loop.**

```

.data
point label BYTE
arr BYTE 'abdullah'
start DWORD 0
endd DWORD 0

```

.code

main PROC

mov eax, lengthof arr

mov endd, eax

L1:

mov esi, start

mov al, [point + esi]

SUB al, 020h

mov [point + esi], al

ADD start, 1

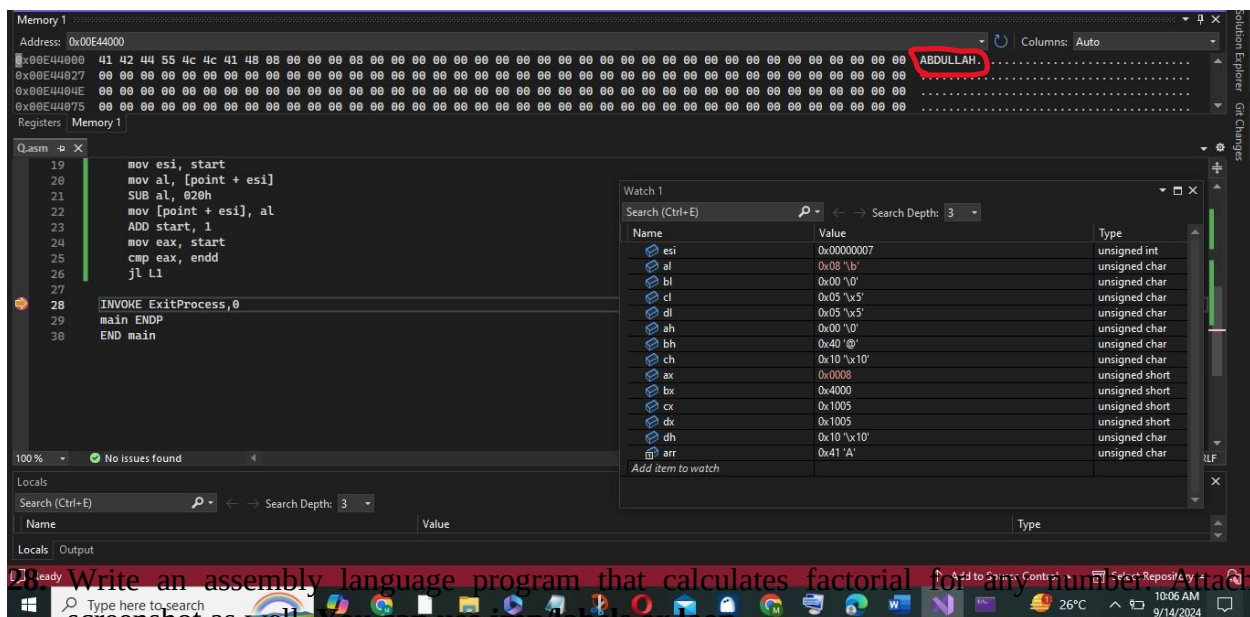
mov eax, start

cmp eax, endd

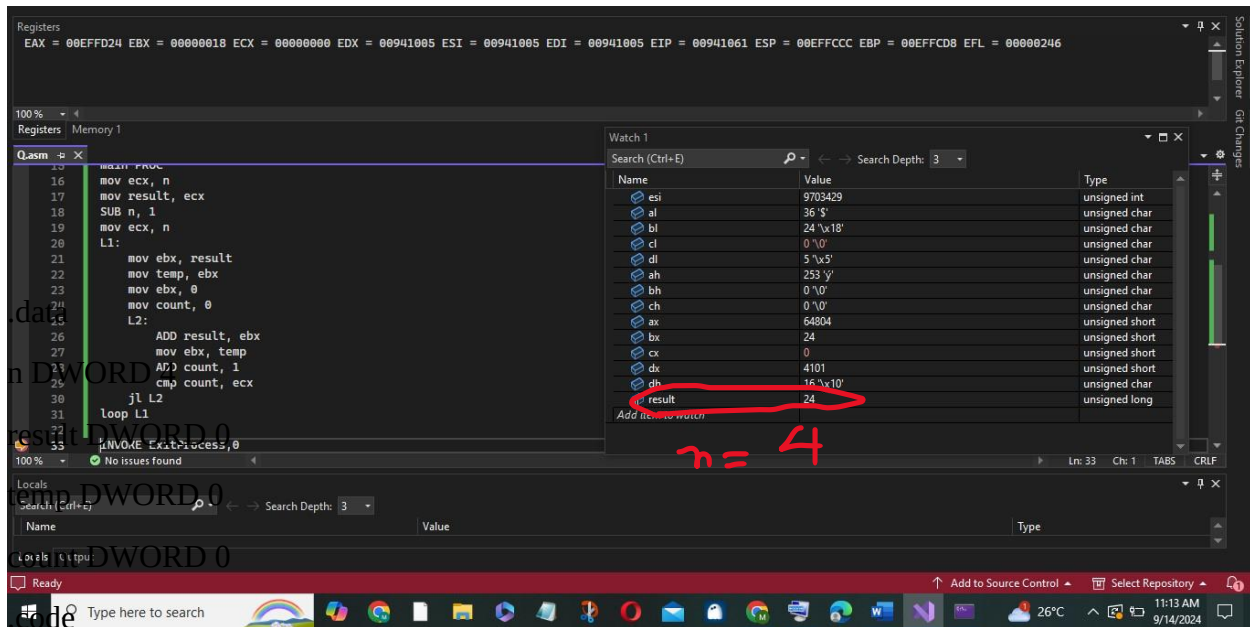
j1 L1

main ENDP

END main



28 ready Write an assembly language program that calculates factorial for any number. Attach screenshot as well. You can use jmp/labels or loop.



main PROC

mov ecx, n

mov result, ecx

SUB n, 1

mov ecx, n

L1:

mov ebx, result

mov temp, ebx

mov ebx, 0

mov count, 0

L2:

ADD result, ebx

mov ebx, temp

ADD count, 1

cmp count, ecx

jl L2

loop L1

INVOKE ExitProcess,0

main ENDP

END main